

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М. А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)

ОТЧЕТ
ЛАБОРАТОРНАЯ РАБОТА №6

Разбиение IP-сетей на подсети

Руководитель

подпись, дата

Панков А. В.

Исполнитель,
группа ИКПИ-32

подпись, дата

Филимонов Д.Е.

Санкт-Петербург 2025

Цель работы

Изучение методов разделения IP-сетей на подсети с использованием классического подхода и технологии VLSM (Variable Length Subnet Mask), а также автоматизация расчетов с помощью языка программирования Python.

Задачи

Разделить сеть 10.2.0.0/16 на подсети согласно требованиям варианта 2:

Задание 1: Создать подсети для **5** и **103** узлов.

Задание 2: Создать подсети для **2000**, **100** и **20** узлов.

Реализовать алгоритм расчета масок подсетей и распределения IP-адресов.

Сравнить эффективность базового метода (без VLSM) и метода VLSM.

Автоматизировать процесс генерации подсетей с помощью Python.

Ход работы:

Лабораторная работа 6.1 Код программы

```
import math
def
calculate_subnet_bits(subnets):
    return math.ceil(math.log2(subnets))
def
decimal_to_ip(decimal):
    return f"{{(decimal >> 24) & 0xFF}}.{{(decimal >> 16) &
0xFF}}.{{(decimal >> 8) & 0xFF}}.{{decimal & 0xFF}}"
    def generate_subnets(base_ip, mask_bits,
required_subnets, num_to_show=5):
        ip_parts = list(map(int, base_ip.split('.')))        network =
(ip_parts[0] << 24) + (ip_parts[1] << 16) + (ip_parts[2]
<< 8) + ip_parts[3]
        subnet_bits =
calculate_subnet_bits(required_subnets)        new_mask =
mask_bits + subnet_bits        step = 2 ** (32 - new_mask)

        # Формирование маски        mask_str = "255.255."        if
new_mask <= 24:            third_octet = (0xFF << (8 - (new_mask -
16))) & 0xFF if new_mask > 16 else 0            mask_str +=
f"{{third_octet}}.0"        else:
            fourth_octet = (0xFF << (32 - new_mask)) & 0xFF
mask_str += f"255.{{fourth_octet}}"
            subnets = []            for i
in range(num_to_show):
                current = network + i * step
broadcast = current + step - 1
first_host = current + 1            last_host
= broadcast - 1

            subnets.append({
                "network": decimal_to_ip(current),
                "mask": mask_str,
                "hosts": f"{{decimal_to_ip(first_host)}} -
{{decimal_to_ip(last_host)}}",
                "broadcast": decimal_to_ip(broadcast)
            })        return
subnets, new_mask
def print_subnet_table(title,
subnets):    print(f"\n=== {title}
===")    print(f"| {'Подсеть':<8} |
{'Адрес сети':<12} | {'Маска':<16} |
{'Диапазон узлов':<30} |
{'Broadcast':<20} |")    print("|-----
-----|-----|-----|-----
|-----|-----|-----|-----")
```

```

-----|")      for idx,
subnet in enumerate(subnets, 1):
    print(f"| {idx:<8} | {subnet['network']:<12} |
{subnet['mask']:<16} | {subnet['hosts']:<30} |
{subnet['broadcast']:<20}
|")

# Общая информация print("\n### Разбиение сети
10.2.0.0/16 ###") print("| Сеть          | Сетевая часть |
Узловая часть |") print("|-----|-----|
|-----|") print("| 10.2.0.0/16| 10.2
| 0.0-255.255 |") print("\nИсходная маска:
255.255.0.0")

# Задание 1: 14 и 158 подсетей subnets_14, mask_14 =
generate_subnets("10.2.0.0", 16, 14) print_subnet_table("Задание 1:
14 подсетей (первые 5)", subnets_14)
subnets_158, mask_158 = generate_subnets("10.2.0.0", 16, 158)
print_subnet_table("Задание 1: 158 подсетей (первые 5)", subnets_158)

# Задание 2: 2 и 100 подсетей subnets_2, mask_2 =
generate_subnets("10.2.0.0", 16, 2) print_subnet_table("Задание
2: 2 подсети (все)", subnets_2[:2])
subnets_100, mask_100 = generate_subnets("10.2.0.0", 16, 100)
print_subnet_table("Задание 2: 100 подсетей (первые 5)", subnets_100)

```

Вывод программы:

Сеть	Сетевая часть	Узловая часть
10.2.0.0/16	10.2	0.0-255.255

Исходная маска: 255.255.0.0

=== Задание 1: 14 подсетей (первые 5) ===

Подсеть	Адрес сети	Маска	Диапазон узлов	Broadcast
1	10.2.0.0	255.255.240.0	10.2.0.1 – 10.2.15.254	10.2.15.255
2	10.2.16.0	255.255.240.0	10.2.16.1 – 10.2.31.254	10.2.31.255
3	10.2.32.0	255.255.240.0	10.2.32.1 – 10.2.47.254	10.2.47.255
4	10.2.48.0	255.255.240.0	10.2.48.1 – 10.2.63.254	10.2.63.255
5	10.2.64.0	255.255.240.0	10.2.64.1 – 10.2.79.254	10.2.79.255

=== Задание 1: 158 подсетей (первые 5) ===

Подсеть	Адрес сети	Маска	Диапазон узлов	Broadcast
1	10.2.0.0	255.255.255.0	10.2.0.1 – 10.2.0.254	10.2.0.255
2	10.2.1.0	255.255.255.0	10.2.1.1 – 10.2.1.254	10.2.1.255
3	10.2.2.0	255.255.255.0	10.2.2.1 – 10.2.2.254	10.2.2.255
4	10.2.3.0	255.255.255.0	10.2.3.1 – 10.2.3.254	10.2.3.255
5	10.2.4.0	255.255.255.0	10.2.4.1 – 10.2.4.254	10.2.4.255

=== Задание 2: 2 подсети (все) ===

Подсеть	Адрес сети	Маска	Диапазон узлов	Broadcast
1	10.2.0.0	255.255.128.0	10.2.0.1 – 10.2.127.254	10.2.127.255
2	10.2.128.0	255.255.128.0	10.2.128.1 – 10.2.255.254	10.2.255.255

=== Задание 2: 100 подсетей (первые 5) ===

Подсеть	Адрес сети	Маска	Диапазон узлов	Broadcast
1	10.2.0.0	255.255.254.0	10.2.0.1 – 10.2.1.254	10.2.1.255
2	10.2.2.0	255.255.254.0	10.2.2.1 – 10.2.3.254	10.2.3.255
3	10.2.4.0	255.255.254.0	10.2.4.1 – 10.2.5.254	10.2.5.255
4	10.2.6.0	255.255.254.0	10.2.6.1 – 10.2.7.254	10.2.7.255

Рис. 1 – Результаты программы 6.1

Код программы

6

```

-----|-----|")
for idx, subnet in enumerate(subnets, 1):
print(f"| {idx:<8} | {subnet['network']:<15} |
{subnet['mask']:<15} | {subnet['hosts']:<30} |
{subnet['broadcast']:<20} |")

# Настройки для варианта 2 (y=2)
base_network = "10.2.0.0"
task1_hosts = [10, 85] task2_hosts
= [14, 400]

# Базовый способ print("\n### Базовый способ
разделения ###") for hosts in [2, 300]:
    subnets = generate_subnets(base_network, [hosts])
    print_subnet_table(f"Разделение на {hosts} узлов", subnets)

# Задание 1 (10, 85 узлов) subnets_task1 =
generate_subnets(base_network, task1_hosts)
print_subnet_table("Задание 1: 10, 85 узлов", subnets_task1)

# Задание 2 (14, 400 узлов) subnets_task2 =
generate_subnets(base_network, task2_hosts)
print_subnet_table("Задание 2: 14, 400 узлов", subnets_task2)
# VLSM print("\n### Способ VLSM ###") vlsm_subnets_task1 =
generate_subnets(base_network, task1_hosts, is_vlsm=True)
print_subnet_table("Задание 1 (10, 85 узлов)", vlsm_subnets_task1)
vlsm_subnets_task2 = generate_subnets(base_network, task2_hosts,
is_vlsm=True) print_subnet_table("Задание 2 (14, 400 узлов)",
vlsm_subnets_task2)

```

Вывод программы

```

### Базовый способ разделения ###

=== Разделение на 2 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Широковещательный адрес |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.252 | 10.2.0.1 – 10.2.0.2 | 10.2.0.3 |

=== Разделение на 300 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Широковещательный адрес |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.254.0 | 10.2.0.1 – 10.2.1.254 | 10.2.1.255 |

=== Задание 1: 10, 85 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Широковещательный адрес |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.240 | 10.2.0.1 – 10.2.0.14 | 10.2.0.15 |
| 2 | 10.2.0.16 | 255.255.255.128 | 10.2.0.17 – 10.2.0.142 | 10.2.0.143 |

=== Задание 2: 14, 400 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Широковещательный адрес |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.240 | 10.2.0.1 – 10.2.0.14 | 10.2.0.15 |
| 2 | 10.2.0.16 | 255.255.254.0 | 10.2.0.17 – 10.2.2.14 | 10.2.2.15 |

### Способ VLSM ###

=== Задание 1 (10, 85 узлов) ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Широковещательный адрес |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.128 | 10.2.0.1 – 10.2.0.126 | 10.2.0.127 |
| 2 | 10.2.0.128 | 255.255.255.240 | 10.2.0.129 – 10.2.0.142 | 10.2.0.143 |

=== Задание 2 (14, 400 узлов) ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Широковещательный адрес |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.254.0 | 10.2.0.1 – 10.2.1.254 | 10.2.1.255 |
| 2 | 10.2.2.0 | 255.255.255.240 | 10.2.2.1 – 10.2.2.14 | 10.2.2.15 |
PS C:\Users\anban\OneDrive\Рабочий стол\zolo>

```

Рис. 2 – Вывод программы 6.2

Лабораторная работа 6.3

Код программы

```
import math
def
calculate_host_bits(hosts):
    host_bits = math.ceil(math.log2(hosts + 2)) # +2 для сети и
broadcast
    return host_bits
def
get_mask(host_bits):
    mask_bits = 32 - host_bits    mask = (0xFFFFFFFF <<
(32 - mask_bits)) & 0xFFFFFFFF    return (
        (mask >> 24) & 0xFF,
        (mask >> 16) & 0xFF,
        (mask >> 8) & 0xFF,      mask
        & 0xFF
    )
def
decimal_to_ip(decimal):
    return f"{{(decimal >> 24) & 0xFF}}.{{(decimal >> 16) &
0xFF}}.{{(decimal >> 8) & 0xFF}}.{{decimal & 0xFF}}"
    def generate_subnets(base_ip, host_counts,
is_vlsm=False):
        ip_parts = list(map(int, base_ip.split('.')))    network =
(ip_parts[0] << 24) + (ip_parts[1] << 16) + (ip_parts[2]
<< 8) + ip_parts[3]
        if
is_vlsm:
            host_counts = sorted(host_counts, reverse=True)
            subnets = []
current = network    for
hosts in host_counts:
            host_bits = calculate_host_bits(hosts)
mask = get_mask(host_bits)
            mask_str = f"{{mask[0]}}.{{mask[1]}}.{{mask[2]}}.{{mask[3]}}"
block_size = 2 ** host_bits

            broadcast = current + block_size - 1
first_host = current + 1    last_host =
broadcast - 1

            subnets.append({
                "network": decimal_to_ip(current),
                "mask": mask_str,
                "hosts": f"{{decimal_to_ip(first_host)}} -
{{decimal_to_ip(last_host)}}",
                "broadcast": decimal_to_ip(broadcast),
                "size": hosts
            })
current += block_size
```

```

        return
subnets
def print_subnet_table(title,
subnets):
    print(f"\n=== {title} ===")    print(f"| {'Подсеть':<8} | {'Адрес
сети':<15} | {'Маска':<15} | {'Диапазон узлов':<30} |
{'Broadcast':<20} |")    print("|-----|-----|-----
-----|-----|
-----|-----|")
for idx, subnet in enumerate(subnets, 1):
    print(f"| {idx:<8} | {subnet['network']:<15} |
{subnet['mask']:<15} | {subnet['hosts']:<30} |
{subnet['broadcast']:<20} |")

# Настройки для варианта 2 base_network = "10.2.0.0"
task1_hosts = [5, 103]    # Задание 1: 5 и 103 узла
task2_hosts = [2000, 100, 20] # Задание 2: 2000, 100, 20 узлов

# Базовое разделение (без VLSM)
print("\n### Базовое разделение ###") for
hosts in task1_hosts:
    subnets = generate_subnets(base_network, [hosts])
print_subnet_table(f"Задание 1: {hosts} узлов", subnets)
for hosts in
task2_hosts:
    subnets = generate_subnets(base_network, [hosts])
print_subnet_table(f"Задание 2: {hosts} узлов", subnets)

# VLSM (оптимизированное разделение)
print("\n### VLSM (оптимизированное) ###")
vlsm_task1 = generate_subnets(base_network, task1_hosts, is_vlsm=True)
print_subnet_table("Задание 1 (5, 103 узлов)", vlsm_task1)

vlsm_task2 = generate_subnets(base_network, task2_hosts, is_vlsm=True)
print_subnet_table("Задание 2 (2000, 100, 20 узлов)", vlsm_task2)

```

Вывод программы

```

### Базовое разделение ###

=== Задание 1: 5 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Broadcast |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.248 | 10.2.0.1 - 10.2.0.6 | 10.2.0.7 |

=== Задание 1: 103 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Broadcast |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.128 | 10.2.0.1 - 10.2.0.126 | 10.2.0.127 |

=== Задание 2: 2000 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Broadcast |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.248.0 | 10.2.0.1 - 10.2.7.254 | 10.2.7.255 |

=== Задание 2: 100 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Broadcast |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.128 | 10.2.0.1 - 10.2.0.126 | 10.2.0.127 |

=== Задание 2: 20 узлов ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Broadcast |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.224 | 10.2.0.1 - 10.2.0.30 | 10.2.0.31 |

### VLSM (оптимизированное) ###

=== Задание 1 (5, 103 узлов) ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Broadcast |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.255.128 | 10.2.0.1 - 10.2.0.126 | 10.2.0.127 |
| 2 | 10.2.0.128 | 255.255.255.248 | 10.2.0.129 - 10.2.0.134 | 10.2.0.135 |

=== Задание 2 (2000, 100, 20 узлов) ===
| Подсеть | Адрес сети | Маска | Диапазон узлов | Broadcast |
|-----|-----|-----|-----|-----|
| 1 | 10.2.0.0 | 255.255.248.0 | 10.2.0.1 - 10.2.7.254 | 10.2.7.255 |
| 2 | 10.2.8.0 | 255.255.255.128 | 10.2.8.1 - 10.2.8.126 | 10.2.8.127 |
| 3 | 10.2.8.128 | 255.255.255.224 | 10.2.8.129 - 10.2.8.158 | 10.2.8.159 |

```

Рис. 3 – Результаты программы 6.3

Вывод

Результаты разделения:

Для Задания 1 подсети успешно созданы:

Подсеть на **103 узла**: 10.2.0.0/25 (маска 255.255.255.128).

Подсеть на **5 узлов**: 10.2.0.128/29 (маска 255.255.255.248).

Для Задания 2:

Подсеть на **2000 узлов**: 10.2.0.0/21 (маска 255.255.248.0).

Подсеть на **100 узлов**: 10.2.8.0/25 (маска 255.255.255.128).

Подсеть на **20 узлов**: 10.2.8.128/27 (маска 255.255.255.224).

Эффективность VLSM:

Метод VLSM позволил сократить потери IP-адресов за счет гибкого распределения подсетей по убыванию требований. Например, для **2000 узлов** использована маска /21, а оставшееся пространство оптимально разделено для меньших подсетей.

Автоматизация:

Реализованная программа на Python успешно генерирует подсети, выводит таблицы с деталями (сеть, маска, диапазон узлов, broadcast) и позволяет сравнивать разные подходы.

Трудности и решения:

Проблема с расчетом масок для нестепеней двойки решена через использование функции `math.ceil()`.

Для обработки больших подсетей (например, на **2000 узлов**) потребовалась корректировка шага смещения адресов.

Итог: Работа позволила закрепить навыки проектирования подсетей, оптимизировать использование адресного пространства и создать инструмент для автоматизации расчетов.